ESERCITAZIONE FINALE

Programmazione ad Oggetti con C# 8

Progetto Console - Sistema di Gestione Flotta Veicoli

1. Descrizione del Progetto

Il candidato è invitato a progettare e sviluppare, in C# 8, un'applicazione console che simuli un sistema di gestione di una flotta di veicoli a noleggio. Il contesto applicativo è volutamente lasciato aperto: il candidato potrà arricchire il dominio con dettagli aggiuntivi, nomi di fantasia, funzionalità extra e qualsiasi scelta progettuale ritenga opportuna, purché vengano rispettati i requisiti minimi descritti nelle sezioni seguenti.

Nota	Non esiste un'unica soluzione corretta. La qualità della progettazione, la coerenza delle scelte architetturali e la leggibilità del codice saranno valutate al pari della correttezza funzionale.

2. Scenario Applicativo

L'azienda "FleetSoft" ha bisogno di un sistema software per gestire la propria flotta di veicoli a noleggio. La flotta è composta da veicoli di categoria diversa (automobili, motocicli, furgoni), ciascuno con caratteristiche proprie. Il sistema deve consentire di:

- Registrare i veicoli disponibili nella flotta.
- Effettuare il noleggio di un veicolo per un numero di giorni specificato, calcolando il costo totale.
- Restituire un veicolo al termine del noleggio, rendendolo nuovamente disponibile.
- Ricevere notifiche (eventi) quando un veicolo viene noleggiato o restituito.
- Visualizzare l'elenco dei veicoli con il loro stato corrente.

Il candidato è libero di estendere lo scenario con ulteriori funzionalità (es. sistema di sconti, storico noleggi, report statistici) senza che ciò sia obbligatorio.

3. Requisiti Tecnici Obbligatori

La soluzione deve obbligatoriamente includere i seguenti elementi. Per ciascuno viene indicato il concetto del corso a cui si riferisce.

Concetto C#	Applicazione nell'esercitazione
Attributi e proprieta	Ogni classe espone dati tramite property con get/set appropriati
Modificatori di accesso	private, protected, public su tutti i membri significativi

Classe vs Istanza	Almeno un membro statico (es. contatore veicoli) vs membri di istanza
Ereditarieta	Auto, Moto, Furgone ereditano dalla classe astratta Veicolo
Incapsulamento	Stato interno privato; accesso solo tramite API pubblica
Polimorfismo	Override di ToString() e metodi virtual/abstract su Veicolo
Eventi generici	event EventHandler<VeicoloEventArgs> sulla classe Flotta
Interfacce	IVeicolo e INoleggiabile implementate dalle classi concrete

3.1 Interfacce

Definire almeno le seguenti interfacce:

- IVeicolo: contratto base per qualsiasi veicolo (es. metodi Accelera e Frena, proprieta' Targa e VelocitaAttuale).
- INoleggiabile: contratto per i veicoli che possono essere noleggiati (metodo Noleggia che restituisce il costo, metodo Restituisci, proprieta' IsDisponibile).

Il candidato puo' aggiungere ulteriori interfacce se lo ritiene utile (es. IAssicurabile, IRevisionabile).

3.2 Gerarchia di Classi ed Ereditarieta

Definire una classe astratta Veicolo che implementi IVeicolo e funga da base per le classi concrete. Implementare almeno tre tipi di veicolo concreti (es. Auto, Moto, Furgone), ciascuno con:

- Attributi privati specifici della categoria (es. numero di porte per Auto, cilindrata per Moto, portata massima per Furgone).
- Proprieta' pubbliche con getter e, dove appropriato, setter privati o protetti.
- Override del metodo ToString() per una rappresentazione testuale descrittiva.
- Implementazione del metodo Noleggia con una tariffa giornaliera differenziata per categoria.

3.3 Incapsulamento e Modificatori di Accesso

Lo stato interno di ogni classe deve essere accessibile solo attraverso l'interfaccia pubblica. Utilizzare correttamente i modificatori private, protected e public. Almeno un membro deve essere dichiarato protected per consentire l'accesso alle classi derivate senza esporlo all'esterno della gerarchia.

3.4 Polimorfismo

La classe Veicolo deve dichiarare almeno un metodo virtual (o abstract). Le classi derivate devono effettuarne l'override con comportamento specifico. Il metodo Noleggia deve essere richiamato in modo polimorfico attraverso una collezione di tipo INoleggiabile, dimostrando che il comportamento corretto viene eseguito in base al tipo effettivo dell'oggetto a runtime.

3.5 Classe Flotta ed Eventi Generici

Definire una classe Flotta che gestisca la collezione di veicoli. La classe deve:

- Mantenere internamente una lista di oggetti di tipo INoleggiabile.
- Esporre un evento generico (EventHandler<T> con un argomento personalizzato che estende EventArgs) scatenato al momento del noleggio e/o della restituzione di un veicolo.
- Esporre metodi per aggiungere veicoli, effettuare e concludere un noleggio, e stampare lo stato corrente della flotta.

3.6 Membri Statici - Classe vs Istanza

Almeno una classe deve contenere un membro statico significativo. Esempi: un contatore statico del numero totale di veicoli istanziati, una costante statica per la tariffa minima di categoria, oppure un metodo statico di factory per la creazione di veicoli con configurazioni predefinite.

3.7 Metodo Main

Il metodo Main deve dimostrare, in modo chiaro e commentato, il funzionamento del sistema: creazione di almeno tre veicoli di categoria diversa, aggiunta alla flotta, sottoscrizione all'evento, esecuzione di almeno due noleggi, visualizzazione dello stato aggiornato e stampa del riepilogo a console.

4. Struttura delle Classi - Diagramma UML

Il diagramma seguente illustra la struttura minima attesa. Il candidato e' libero di aggiungere classi, interfacce o relazioni aggiuntive, purche' i requisiti della sezione 3 siano soddisfatti.

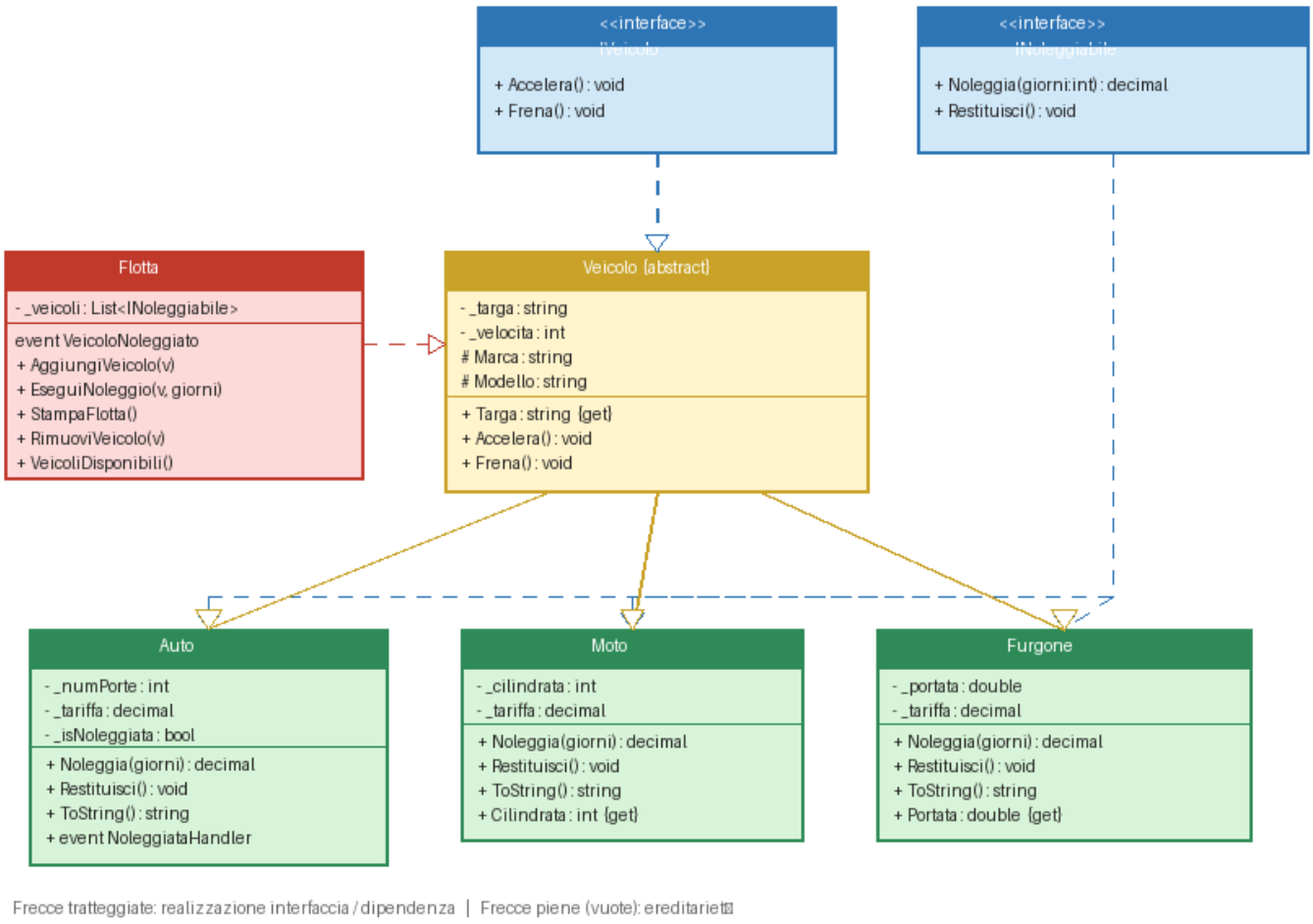

Figura 1 - Diagramma UML della struttura minima (il candidato puo' estenderla liberamente)

5. Esempio di Output Atteso

A titolo puramente indicativo, l'esecuzione del programma potrebbe produrre un output simile al seguente. I valori numerici, le tariffe e i messaggi possono variare in base alle scelte del candidato.

```
=== FleetSoft --- Sistema di Gestione Flotta ===

Veicoli registrati nella flotta:
  [1] Auto     | AB123CD | Ford Focus  | 4 porte | Disponibile
  [2] Moto     | EF456GH | Honda CB500 | 500 cc  | Disponibile
  [3] Furgone  | IL789MN | Fiat Ducato | 1.5 t   | Disponibile

Noleggio veicolo [1] per 5 giorni...
  >> [EVENTO] Noleggio avviato: Ford Focus (AB123CD) - Costo: 225.00 EUR

Noleggio veicolo [2] per 3 giorni...
  >> [EVENTO] Noleggio avviato: Honda CB500 (EF456GH) - Costo: 105.00 EUR

Stato aggiornato della flotta:
  [1] Auto     | AB123CD | Ford Focus  | NOLEGGIATA
  [2] Moto     | EF456GH | Honda CB500 | NOLEGGIATA
  [3] Furgone  | IL789MN | Fiat Ducato | Disponibile
```

```
Totale veicoli istanziati (membro statico): 3
```

6. Modalita' di Consegna

1. Creare una Solution Visual Studio denominata FleetSoft (o nome scelto dal candidato).
2. Il progetto deve essere di tipo Console Application (.NET 6 o superiore, C# 8 come versione del linguaggio target).
3. Il codice deve essere organizzato in file distinti per ciascuna classe e interfaccia.
4. Consegnare la cartella dell'intera Solution compressa in formato .zip con nome: Cognome_Nome_Esercitazione.zip.

7. Suggerimenti

Di seguito alcuni spunti per orientare le scelte progettuali:

- Definire prima le interfacce, poi la classe astratta, infine le classi concrete: questo ordine favorisce un design coerente.
- Per l'evento generico, definire una classe VeicoloEventArgs che estenda EventArgs, contenente le informazioni rilevanti (riferimento al veicolo, costo del noleggio, tipo di operazione).
- Usare il tipo decimal per i valori monetari, evitando errori di arrotondamento tipici di float e double.
- Il polimorfismo puo' essere dimostrato iterando una List<INoleggiabile> e invocando Noleggia su ogni elemento senza conoscerne il tipo concreto.
- I membri statici sono utili per un registro centrale o per definire tariffe base comuni a una categoria di veicoli.

Ricorda	La semplicita' ben progettata vale piu' della complessita' mal strutturata. Un sistema con poche classi, responsabilita' chiare e relazioni corrette sara' valutato positivamente.

Buon lavoro!

Il miglior codice e' quello che gli altri e il tuo futuro io riescono a leggere e capire facilmente.